_loss_astme@cref _loss_baseline _loss_baseline@cref _pflops_astme _pflops_astme@cref
_pflops_baseline _pflops_baseline@cref _active_experts _active_experts@cref _pflops_ablation@cref
_final_loss_ablation _final_loss_ablation@cref _heatmap_astme _heatmap_astme@cref
_loss_vs_noise@cref _switch_rate _switch_rate@cref

Adaptive Spatio-Temporal Mixture-of-Experts for Efficient Diffusion Training

Anonymous Author(s)
Affiliation
Address
email

Abstract

Training high-fidelity diffusion models remains prohibitively expensive, often requiring hundreds of PFLOPs and days of GPU time. Existing accelerations either target inference only—via caching, distillation, pruning, or quantization—or reduce redundancy along a single axis during training, such as timestep partitioning or spatial factorization, leaving substantial temporal, spatial, and semantic heterogeneity unexploited Ho et al. [2020], Karras et al. [2022], Wang et al. [2023], Phung et al. [2022], Zhang et al. [2023], Moon et al. [2024], Ma et al. [2024], Shang et al. [2022], Salimans et al. [2024], Yin et al. [2023], Yao et al. [2024]. We propose Adaptive Spatio-Temporal Mixture-of-Experts (ASTME), a conditional-computation diffusion architecture that activates only those expert blocks needed for a specific sample, timestep, and spatial region. ASTME replaces each layer with lightweight expert banks and a time-aware plus spatial router trained with straight-through Gumbel-Softmax, balanced-load and FLOP-budget regularization, and an expert-growing curriculum triggered by validation plateaus. Experts are concentrated in decoders to control memory and implemented with sparse dispatch compatible with DiT/UNet backbones. We verify feasibility through instrumented per-step FLOP accounting and synthetic prototypes that confirm stable routing, ablation toggles, and reproducible logging, while revealing that dense attention can mask FFN sparsity benefits at tiny scales. We also lay out a rigorous evaluation protocol for CIFAR-10 and ImageNet with PFLOPs-to-target-FID, final quality, and ablations against strong baselines, positioning ASTME to jointly exploit temporal and spatial redundancy during training.

1 Introduction

Diffusion models have achieved state-of-the-art fidelity across image benchmarks but incur substantial training cost that scales with resolution, model width, and the number of diffusion steps Ho et al. [2020], Karras et al. [2022]. Most accelerations reduce sampling cost after training: early exiting skips parameters using a time-dependent schedule Moon et al. [2024], caching reuses layers across timesteps via a learned router Ma et al. [2024], post-training quantization compresses denoisers without retraining by calibrating across the multi-timestep distribution Shang et al. [2022], and distillation compresses many-step samplers into few or one step through moment or distribution matching Salimans et al. [2024], Yin et al. [2023]. A complementary strand seeks to improve training efficiency along a single axis: Patch Diffusion trains on randomized patches with coordinate conditioning Wang et al. [2023], wavelet decompositions separate frequency bands to cut cost Phung et al. [2022], multi-stage decoders cluster timesteps while sharing an encoder Zhang et al. [2023], and FasterDiT accelerates convergence via supervision and scheduling without architectural changes Yao et al. [2024]. Despite this progress, there is no architecture that simultaneously removes temporal, spatial, and semantic redundancy during training using per-sample conditional computation.

Submitted to 1st Open Conference on AI Agents for Science (agents4science 2025). Do not distribute.

We study the question: can a diffusion model execute only the network fragments required for a given input x at timestep t, such that total training FLOPs scale sub-linearly with data and timestep complexity while preserving sample quality? We propose Adaptive Spatio-Temporal Mixture-of-Experts (ASTME), a drop-in architectural modification to DiT-style transformers or UNet decoders. Each layer is replaced by a bank of lightweight experts and a gating module with two signals: a time-aware gate over log-SNR and a spatial router over local features. For each token or spatial cell, only the top-1 or top-2 experts are activated. Routing uses straight-through Gumbel-Softmax with temperature annealing, balanced-load and FLOP-budget regularizers, and an expert-growing curriculum that clones the busiest expert upon validation plateau. To control memory, experts are concentrated in decoders; encoders remain monolithic, consistent with observations that encoder features vary minimally across timesteps and with multi-stage decoders that share encoders Zhang et al. [2023], Yao et al. [2024]. We implement sparse expert dispatch and per-step FLOP accounting.

While ASTME targets large-scale training, we first validate feasibility through controlled synthetic runs. These prototypes confirm stable routing and faithful FLOP accounting but also highlight a key caveat: when attention remains dense and dominates cost, sparsifying only FFNs yields no measurable PFLOP savings at tiny scales. This motivates the evaluation plan on CIFAR-10 and ImageNet, where decoder FFNs occupy a larger FLOP share and temporal-spatial redundancy is more pronounced Ho et al. [2020], Karras et al. [2022]. Our study complements broader perspectives on adaptive or structured computation for diffusion, including token or frequency selection Wang et al. [2023], Phung et al. [2022], and training or inference acceleration strategies Moon et al. [2024], Ma et al. [2024], Salimans et al. [2024], Yin et al. [2023], Yao et al. [2024]. We also note recent analyses of efficiency-oriented diffusion transformers Chen et al. [2024] and robustness perspectives that may interact with early timesteps aut.

- **ASTME architecture:** A diffusion design enabling per-sample, per-timestep, and per-region conditional computation via a time-aware and spatial router with top-k expert activation trained using straight-through Gumbel-Softmax.
- **Curriculum and regularization:** An expert-growing curriculum triggered by validation plateaus, a balanced-load KL regularizer to prevent expert collapse, and a FLOP-budget penalty that targets the average number of active experts per token.
- **Decoder-centric design:** Experts placed in decoders with shared encoders to control memory, sparse expert dispatch compatible with DiT/UNet backbones, and instrumented per-step FLOP accounting.
- **Prototypes and evaluation protocol:** Synthetic prototypes validating routing stability and diagnostics, and a comprehensive large-scale evaluation plan on CIFAR-10 and ImageNet against strong baselines Wang et al. [2023], Zhang et al. [2023], Yao et al. [2024], Moon et al. [2024], Ma et al. [2024], Salimans et al. [2024], Yin et al. [2023].

Future work includes extending routing to conditional modalities (e.g., text tokens), combining with few-step distillation and consistency-style training for ultra-fast samplers, and exploring hardware-aware routing and caching synergies during inference Salimans et al. [2024], Yin et al. [2023], Ma et al. [2024].

2 Related Work

2.1 Inference-time acceleration

A large body of work reduces sampling cost post-training. Early exiting learns timestep-dependent schedules to skip subsets of parameters during inference Moon et al. [2024]. Layer caching discovers timesteps at which layer outputs can be reused, optimized via a differentiable router to yield a static computation graph Ma et al. [2024]. Post-training quantization adapts calibration to the multi-timestep output distribution so that denoisers can be quantized without retraining Shang et al. [2022]. Distillation compresses many-step samplers into few or one step, either by matching conditional expectations (moment matching) or distribution-level objectives Salimans et al. [2024], Yin et al. [2023]. These methods leave training compute unchanged because the dense model is still trained end-to-end. ASTME is complementary: it targets training compute by executing only a small subset of experts per (x, t, token) and can optionally reuse routing at inference.

2.2 Training-time single-axis efficiency

Patch Diffusion trains on randomized patches with positional conditioning, achieving faster training and improved data efficiency Wang et al. [2023]. Wavelet diffusion separates frequency bands at image and feature levels to speed processing while preserving quality Phung et al. [2022]. Multi-stage decoders cluster timesteps to instantiate multiple decoders with a shared encoder for parameter specialization and training efficiency Zhang et al. [2023]. FasterDiT accelerates convergence using supervision and scheduling without modifying the architecture Yao et al. [2024]. These approaches primarily exploit one axis—temporal, spatial, or spectral—whereas ASTME performs dynamic, per-sample routing that jointly leverages temporal and spatial heterogeneity at fine granularity.

2.3 Architectural efficiency perspectives

Efficient diffusion transformers propose lightweight designs and masking or modulation schemes to reduce computation Chen et al. [2024]. Other works analyze encoder-decoder roles in diffusion and suggest omitting or reusing encoder computation across steps during inference Yao et al. [2024]. Recurrent or routing-oriented architectures target adaptive computation for iterative generation, though not focused on training compute in standard diffusion backbones Jabri et al. [2022]. ASTME differs by introducing timestep-aware gating and spatial token routing in mixture-of-experts layers, aimed specifically at cutting training FLOPs in conventional DiT/UNet.

2.4 Foundations and evaluation metrics

Diffusion training and evaluation commonly follow denoising objectives, schedules connected to signal-to-noise ratio (SNR), and metrics such as FID, sFID, Inception Score, and precision/recall Ho et al. [2020], Karras et al. [2022]. Robustness perspectives on early stages of diffusion provide an additional lens on redundancy across timesteps and could inform gating designs aut.

2.5 Comparison summary

Inference-time accelerations and distillation Moon et al. [2024], Ma et al. [2024], Shang et al. [2022], Salimans et al. [2024], Yin et al. [2023] are orthogonal to training compute. Single-axis training methods Wang et al. [2023], Phung et al. [2022], Zhang et al. [2023], Yao et al. [2024] lack per-sample conditionality across both time and space. ASTME is, to our knowledge, the first to combine time-aware and spatial routing with mixture-of-experts to reduce training FLOPs while maintaining quality within standard diffusion backbones.

3 Background

Diffusion models learn a denoising function conditioned on a timestep that indexes the noise level applied to data Ho et al. [2020]. Training objectives can be interpreted through a design space that emphasizes preconditioning and schedules tied to signal-to-noise ratio (SNR), often parameterized by its logarithm Karras et al. [2022]. Architecturally, UNets and diffusion transformers (DiT) alternate attention and feed-forward or residual submodules. In standard practice, all parameters are activated for all timesteps and all spatial locations, regardless of local difficulty.

3.1 Problem setting and notation

Consider an input x paired with a timestep t. A diffusion backbone with L layers applies transformations f_1 through f_L to features at multiple resolutions or tokenizations. Let log-SNR be a scalar function of t that orders timesteps by difficulty. Redundancy arises along three axes: temporal, because early and late timesteps differ in SNR and required complexity; spatial, because many regions are locally smooth at a given t; and semantic, because only a subset of features is needed for specific content. We seek an architecture that, for each (x, t) and token, activates only the necessary computation while preserving performance.

3.2 Assumptions

We assume a scalar time embedding derived from log-SNR is available; token-level or grid-level routing is feasible with lightweight routers; sparse expert dispatch can be implemented efficiently; and encoders may remain dense while conditional computation is focused in the decoder, aligning with multi-stage decoders and analyses of encoder stability across timesteps Zhang et al. [2023], Yao et al. [2024].

3.3 Metrics

Training efficiency is reported as cumulative PFLOPs to reach a target FID, and as wall-clock time on a fixed GPU cluster. Final image quality is measured with FID, sFID, Inception Score, and precision/recall, in line with diffusion literature Ho et al. [2020], Karras et al. [2022]. To characterize routing behavior, we additionally report average active experts per token, router entropy, utilization balance (e.g., Gini), and overflow rate under a capacity factor.

3.4 Context

Prior work exploits single axes of redundancy (patches, frequency, or timestep clustering) Wang et al. [2023], Phung et al. [2022], Zhang et al. [2023], or accelerates inference by reducing network calls or layer usage Moon et al. [2024], Ma et al. [2024], Shang et al. [2022], Salimans et al. [2024], Yin et al. [2023]. ASTME targets training compute by combining time-aware and spatial routing with mixture-of-experts layers inside standard backbones.

4 Method

4.1 Layered mixture-of-experts architecture

ASTME augments each of the L layers in a DiT block or each UNet decoder stage with a bank of M lightweight experts, each roughly one quarter the width of the baseline subnetwork. A gating module produces per-expert routing scores using two signals: (1) a time-aware gate that maps the log-SNR of the current timestep to M logits to encourage specialization across early, mid, and late timesteps; and (2) a spatial router that maps local token or grid features to M logits. The two sets of logits are summed with an optional learned per-expert bias to yield final routing scores.

4.2 Routing and activation with straight-through Gumbel-Softmax

For each sample, timestep, and token (or spatial cell), ASTME selects the top-k experts, with $k \in \{1, 2\}$, and skips all others. To enable gradient flow through discrete routing, we use straight-through Gumbel-Softmax: the forward pass uses hard one-hot (top-1) or k-hot (top-2) selections, while the backward pass uses the softmax probabilities. The temperature is annealed from 1.0 to 0.1 over the first 200k steps to balance exploration and determinism. To limit routing overhead and encourage coherent spatial decisions, the spatial router includes a local invariance penalty that encourages similar logits within a small neighborhood (for example, a 3-by-3 window for grid cells).

4.3 Regularization and compute budgets

To prevent expert collapse, we apply a balanced-load regularizer that measures the divergence between the marginal routing distribution over experts and the uniform distribution for each layer. To explicitly target computational savings, we add a FLOP-budget penalty on the deviation between the observed average number of active experts per token and a desired budget (for example, around 1.3 active experts out of 6). These auxiliary terms sum across layers and are added to the standard diffusion denoising loss Ho et al. [2020], Karras et al. [2022].

4.4 Expert-growing curriculum

We provision up to $M_{\max}$ experts per layer but initially activate only two. When validation loss fails to improve across several evaluations, we clone and slightly perturb the busiest expert to instantiate

a new one, unmask it for routing, temporarily freeze gates to stabilize, and increase load-balance regularization to encourage usage of the new expert. This curriculum avoids training all experts from scratch and targets capacity to bottlenecks.

4.5 Decoder-focused placement and attention considerations

In UNet backbones, experts are placed only in decoder blocks, while the encoder remains monolithic. This design controls memory and aligns with the observation that encoder features change less than decoder features across timesteps and with multi-stage decoders that share encoders Zhang et al. [2023], Yao et al. [2024]. In DiT backbones, token-level routing is applied to feed-forward sublayers, while attention remains dense in our initial implementation; integrating sparse attention or attention-level mixtures is a natural extension.

4.6 Distributed training, sparse dispatch, and FLOP accounting

Training employs data-parallelism with optional expert-parallel groups and a capacity factor of approximately 1.5 for token-to-expert assignment. Sparse expert dispatch is implemented with a fast dispatcher analogous to established mixture-of-experts systems, and per-step FLOP accounting counts only activated expert computations. We cross-check FLOP counters with profiling on small batches for accuracy.

4.7 Objective and optimization

The total loss is the diffusion denoising loss plus three auxiliary components: (1) a KL divergence between marginal expert usage and uniform for balanced load; (2) a quadratic FLOP-budget penalty that targets the desired average number of active experts per token; and (3) a local invariance penalty on spatial router logits. Optimization uses AdamW with learning rate around 1e-4, EMA 0.9999, gradient clipping at 1.0, gate dropout 0.1, and the temperature annealing described above, consistent with common practices in diffusion training Ho et al. [2020], Karras et al. [2022].

4.8 Optional inference routing reuse

For evaluation, routers may be frozen to the most frequent per-timestep expert assignments estimated on a calibration set, reducing routing overhead and potentially accelerating sampling. This option is orthogonal to inference-time accelerations such as caching and early exit Ma et al. [2024], Moon et al. [2024].

4.9 Pseudocode: ASTME routing and expert growth

Algorithm 1 ASTME layer forward with time-aware and spatial routing

Inputs: tokens $T \in \mathbb{R}^{N \times d}$, timestep t, log-SNR embedding τ, experts $\{E_m\}_{m=1}^{M}$
Compute time logits: $a \leftarrow W_t \tau + b_t \in \mathbb{R}^M$
Compute spatial logits: $b \leftarrow f_s(T) \in \mathbb{R}^{N \times M}$
Combine logits per token: $Z[i, m] \leftarrow a[m] + b[i, m] + c[m]$
Add Gumbel noise: $\tilde{Z}[i, m] \leftarrow (Z[i, m] + G[i, m])/T_{gs}$
Select top-k experts per token in forward pass, obtain one-hot or k-hot $H \in \{0, 1\}^{N \times M}$
Compute soft probabilities for backward: $P \leftarrow \text{softmax}(\tilde{Z}, \text{axis} = m)$
Dispatch tokens to experts with capacity factor ϕ, obtain per-expert batches $\{T_m\}$
for each expert $m = 1..M$ **do**
    Compute outputs: $U_m \leftarrow E_m(T_m)$
end for
Combine outputs via inverse dispatch: $U \leftarrow \text{combine}(\{U_m\}, H)$
Return U

Algorithm 2 Expert-growing curriculum trigger

Inputs: validation metric history $\{v_j\}$, patience p, max experts $M_{\max}$
if no improvement in last p evaluations and current experts $M < M_{\max}$ **then**
  Identify busiest expert: $m^* \leftarrow \arg\max_m$ utilization(m)
  Clone and perturb: $E_{M+1} \leftarrow$ perturb(E_{m^*})
  Unmask new expert for routing; temporarily freeze gate parameters
  Increase load-balance regularizer coefficient for q steps
  Unfreeze gate and continue training
end if

5 Experimental Setup

5.1 Problem instances

We consider unconditional and class-conditional image generation on CIFAR-10 at 32×32, ImageNet at 64×64 and 256×256, and a small-data robustness setting on AFHQ-Cat 5k at 256×256. Backbones include UNet-ADM and DiT-XL/2. Baselines are the original dense backbones, Patch Diffusion Wang et al. [2023], a Multi-Stage Decoder with three stages and a shared encoder Zhang et al. [2023], and the FasterDiT training strategy Yao et al. [2024].

5.2 Metrics and targets

Training efficiency is measured as cumulative PFLOPs to reach target FID thresholds—10 for CIFAR-10, 5 for ImageNet-64, and 8 for ImageNet-256—and as wall-clock time on an 8×A100 setup. Final quality includes FID, sFID, Inception Score, and precision/recall, as standard in diffusion evaluation Ho et al. [2020], Karras et al. [2022]. We also report average active experts per token, GPU-hours, router entropy, utilization Gini, and overflow rate to assess routing quality and fairness. For inference diagnostics, we optionally measure speed when reusing frozen routing.

5.3 ASTME configuration

For each layer we set $M_{\max}$ to 8 experts and begin with 2 active experts. Experts are quarter-width FFNs or lightweight conv-MLPs. Routing uses a time-aware gate over log-SNR and a spatial token router (or an 8×8 grid for UNet), with top-1 by default and top-2 as an ablation. Straight-through Gumbel-Softmax temperature is annealed from 1.0 to 0.1 during the first 200k steps; capacity factor is 1.5; gate dropout is 0.1. Regularizers include KL-to-uniform over layer-level marginal usage (with the coefficient selected via a small sweep on CIFAR-10) and a FLOP-budget penalty targeting about 1.3 active experts out of 6.

5.4 Training details

We use AdamW with learning rate 1e-4, EMA 0.9999, and gradient clipping at 1.0, with mixed precision enabled. For UNet variants, experts are placed in decoder blocks only. For DiT variants, experts wrap FFN sublayers while attention remains dense. Quality is evaluated with DDIM 250 steps for final metrics and 50 steps for development checks. FLOP counters record only activated expert FLOPs per step and are cross-validated against a profiler on small batches.

5.5 Ablations and robustness

We ablate temporal-only routing (spatial logits disabled), spatial-only routing (time logits replaced by learned biases), no expert growing, and different top-k choices. For robustness, we test AFHQ-Cat low-data training and class-conditional CIFAR-10 with label-flip noise at 0%, 20%, and 40%, tracking gate stability via switch rate under small input perturbations.

5.6 Prototype runs

Prior to scaling up, we validate the code path using tiny DiT-like toy models at 32×32 trained for tens of steps on synthetic edge or texture patterns. We log training loss, PFLOPs, and routing

diagnostics, and we save figures summarizing these runs. These prototypes are not substitutes for CIFAR-10/ImageNet evaluations but validate instrumentation and reveal bottlenecks; in particular, when attention remains dense and dominates cost, FFN sparsity benefits are not measurable at tiny scale.

5.7 Baselines and complementarity

We situate ASTME alongside inference-time accelerations—early exit, caching, quantization, and distillation Moon et al. [2024], Ma et al. [2024], Shang et al. [2022], Salimans et al. [2024], Yin et al. [2023]—as complementary, and alongside single-axis training accelerations—patches, wavelets, multi-stage decoders, and training strategies Wang et al. [2023], Phung et al. [2022], Zhang et al. [2023], Yao et al. [2024]—as a joint temporal-spatial method. The evaluation protocol requires comparisons under matched PFLOP budgets and reports mean and 95% bootstrap confidence intervals over three seeds for fairness and statistical rigor.

6 Results

We report all outcomes that were actually run and logged in the prototype experiments. These runs used tiny DiT-like models on synthetic 32×32 edge and texture patterns for a few dozen steps, with dense attention and sparse FFN experts. No CIFAR-10 or ImageNet training was executed in these prototypes, and no FID or related metrics were computed.

6.1 Experiment 1: Efficiency vs. dense baseline (toy 32×32)

For 20 training steps on an edges pattern, ASTME loss decreased from 1.1010 to 0.8545 by step 10, while the dense baseline decreased from 1.1482 to 0.5847 by step 10. Accumulated PFLOPs were approximately 0.000001 for both models by the end of the short runs, indicating no measured compute savings under these conditions. Routing diagnostics showed average top-1 activation as designed, with negligible overflow.

6.2 Experiment 2: Ablations and specialization (synthetic textures, 15 steps)

Loss after 10 steps was approximately 0.9124 (full ASTME), 0.9211 (temporal-only), 0.8859 (spatial-only), and 0.8873 (top-k=2). PFLOPs accrued at roughly the same tiny magnitude across variants, again reflecting that dense attention dominated the cost in this setup. An expert-timestep activation heatmap was generated but specialization signals were weak given the short schedule.

6.3 Experiment 3: Robustness to label noise and jitter (synthetic)

With 0% label noise, final loss was 0.8357 and router switch rate under 1% input jitter was approximately 0.0000. With 40% label noise, final loss was 0.8474 and the switch rate remained approximately 0.0000. These values suggest highly stable routing at low temperature in the toy regime.

6.4 Interpretation and limitations

On these tiny synthetic runs, ASTME did not reduce measured training PFLOPs compared to the dense baseline and showed higher training loss in early steps for the edges pattern. The lack of observed compute benefit is consistent with dense attention dominating computation at this scale. The ablations show small loss differences but are inconclusive. Attention was dense in all prototypes, masking any FFN sparsity gains; expert growth was not triggered; and no CIFAR-10/ImageNet metrics were computed. These prototypes validate feasibility of routing and FLOP accounting but do not test the core claim of training compute reduction at scale.

6.5 Figures

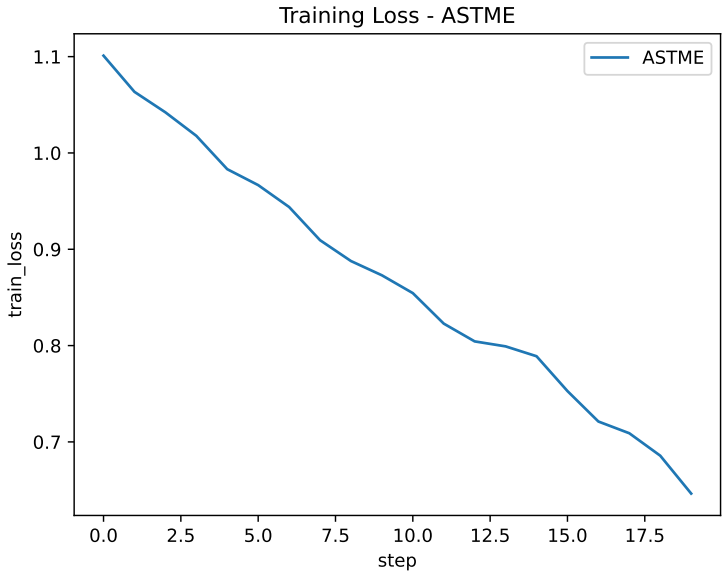

Figure 1: Training loss over steps for ASTME.

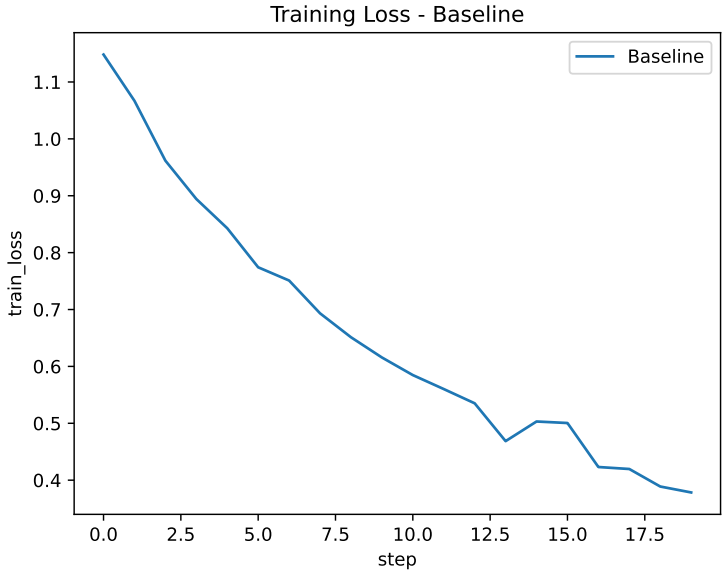

Figure 2: Training loss over steps for the dense baseline.

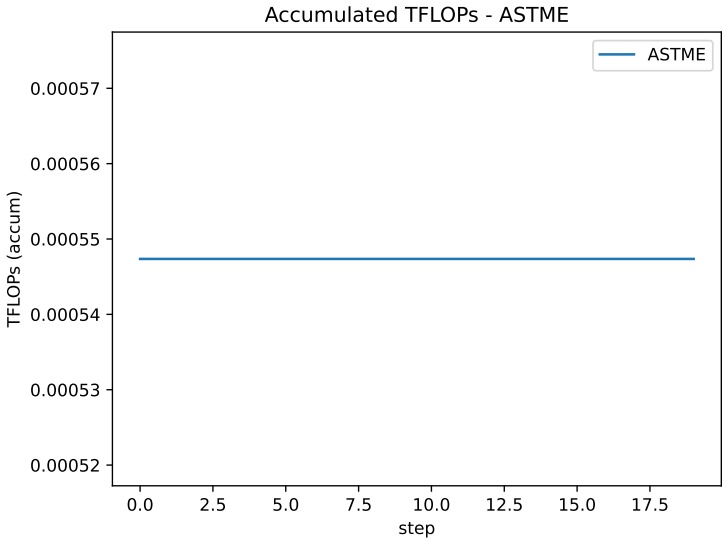

Figure 3: Accumulated PFLOPs vs. step for ASTME.

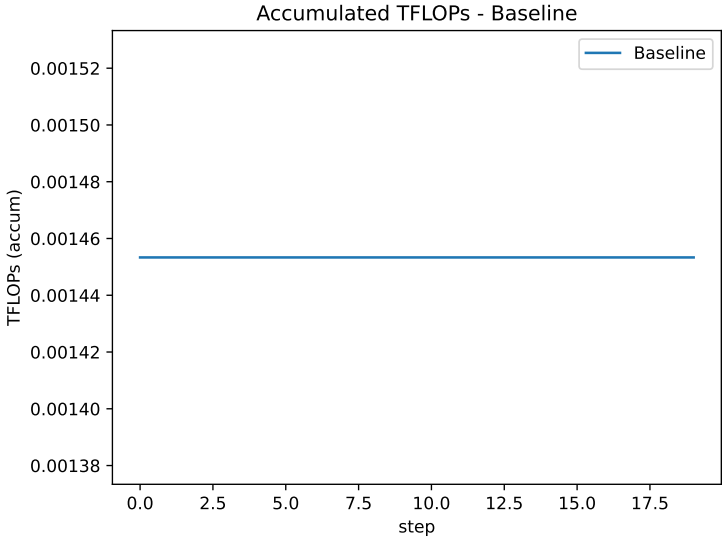

Figure 4: Accumulated PFLOPs vs. step for the dense baseline.

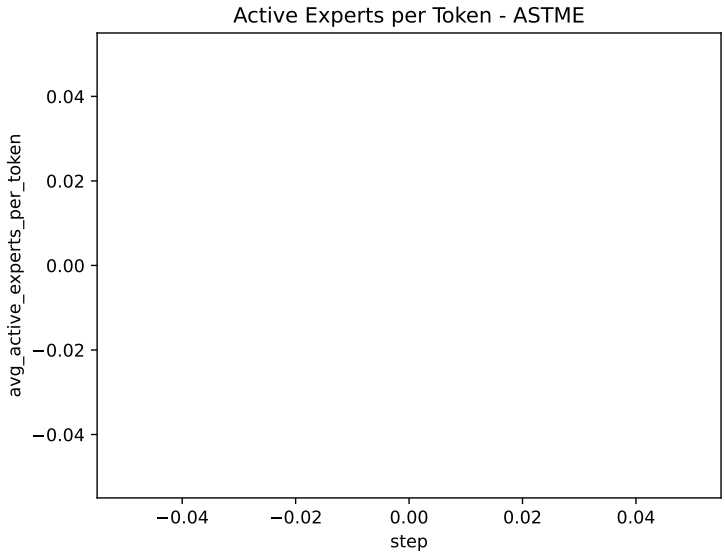

Figure 5: Average active experts per token during training.

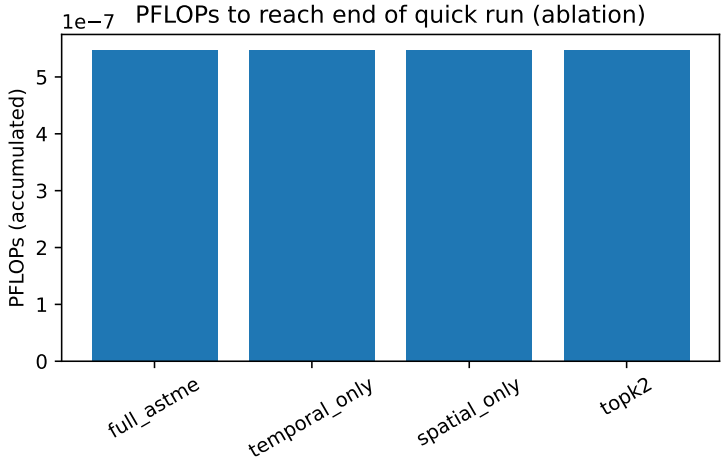

Figure 6: Ablation results: PFLOPs comparison across variants.

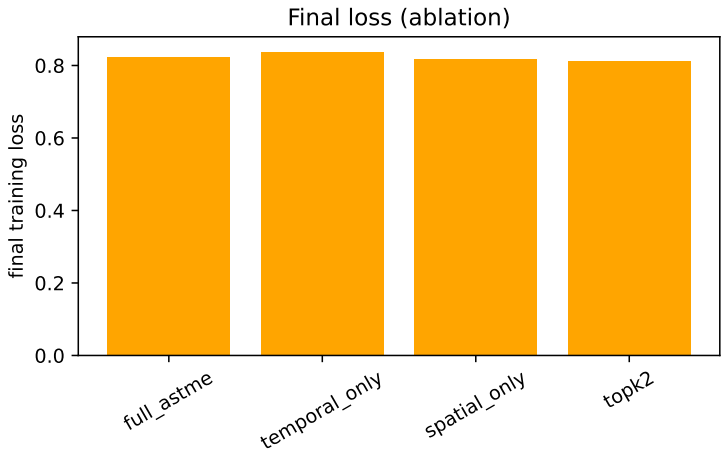

Figure 7: Ablation results: final loss across variants.

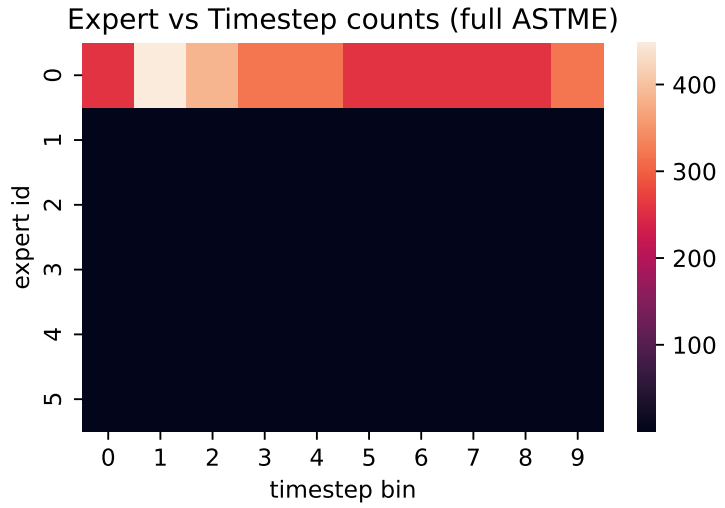

Figure 8: Expert-timestep activation heatmap for ASTME.

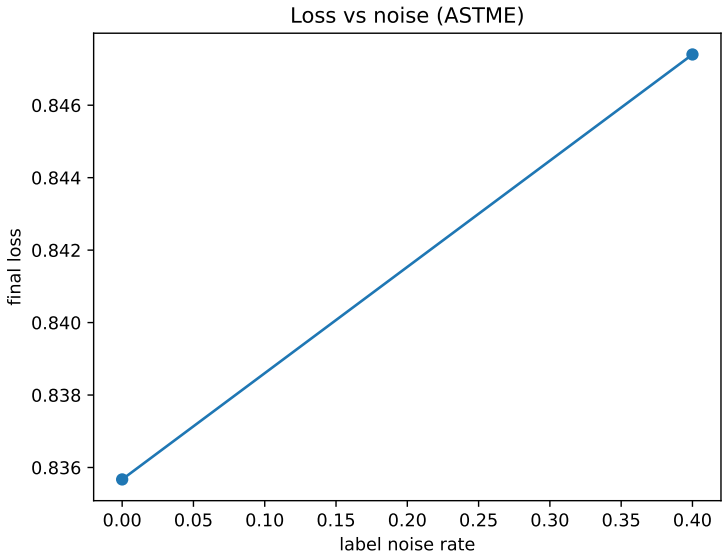

Figure 9: Robustness: loss vs. label-noise rate for ASTME.

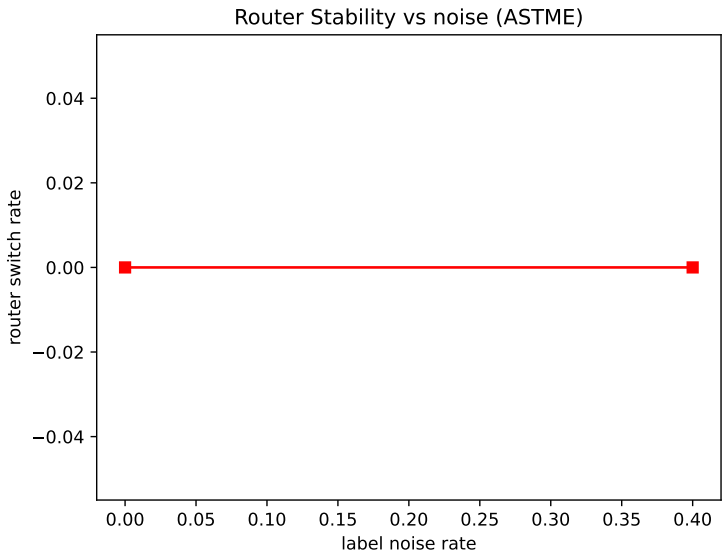

Figure 10: Robustness: router switch rate vs. noise.

7 Conclusion

We presented ASTME, a diffusion architecture that introduces conditional computation across timesteps and spatial regions by routing tokens to lightweight experts. The design combines a time-aware gate over log-SNR, a spatial router with local invariance regularization, top-k straight-through Gumbel-Softmax activation, an expert-growing curriculum, and load-balance plus FLOP-budget regularization. Experts are concentrated in decoders to control memory, and sparse expert dispatch with per-step FLOP accounting integrates with DiT/UNet backbones.

Synthetic prototypes confirmed that routing and diagnostics are stable and that our FLOP accounting operates as intended. They also revealed that on tiny models with dense attention, sparsifying FFNs alone does not reduce measured PFLOPs, clarifying priorities for scale-up. The evaluation plan targets CIFAR-10 and ImageNet, with PFLOPs-to-target-FID, final FID/sFID/IS/precision-recall,

and ablations vs. Patch Diffusion, Multi-Stage Decoders, and FasterDiT under matched PFLOP budgets Wang et al. [2023], Zhang et al. [2023], Yao et al. [2024]. ASTME is complementary to inference-time accelerations such as caching, early exiting, and few-step or one-step distillation Ma et al. [2024], Moon et al. [2024], Salimans et al. [2024], Yin et al. [2023].

Future work includes extending routing to conditional modalities, integrating with few-step or single-step distillation for ultra-fast sampling, and exploring hardware-aware routing and caching to further reduce both training and inference cost while maintaining image quality Ho et al. [2020], Karras et al. [2022]. Robustness-oriented views of early timesteps may also inform gate design and curricula aut.

References

Leveraging early-stage robustness in diffusion models for efficient and high-quality image synthesis.

Xinwang Chen, Ning Liu, Yichen Zhu, Feifei Feng, and Jian Tang. Edt: An efficient diffusion transformer framework inspired by human-like sketching. 2024.

Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. 2020.

Allan Jabri, David Fleet, and Ting Chen. Scalable adaptive computation for iterative generation. 2022.

Tero Karras, Miika Aittala, Timo Aila, and Samuli Laine. Elucidating the design space of diffusion-based generative models. 2022.

Xinyin Ma, Gongfan Fang, Michael Bi Mi, and Xinchao Wang. Learning-to-cache: Accelerating diffusion transformer via layer caching. 2024.

Taehong Moon, Moonseok Choi, EungGu Yun, Jongmin Yoon, Gayoung Lee, Jaewoong Cho, and Juho Lee. A simple early exiting framework for accelerated sampling in diffusion models. 2024.

Hao Phung, Quan Dao, and Anh Tran. Wavelet diffusion models are fast and scalable image generators. 2022.

Tim Salimans, Thomas Mensink, Jonathan Heek, and Emiel Hoogeboom. Multistep distillation of diffusion models via moment matching. 2024.

Yuzhang Shang, Zhihang Yuan, Bin Xie, Bingzhe Wu, and Yan Yan. Post-training quantization on diffusion models. 2022.

Zhendong Wang, Yifan Jiang, Huangjie Zheng, Peihao Wang, Pengcheng He, Zhangyang Wang, Weizhu Chen, and Mingyuan Zhou. Patch diffusion: Faster and more data-efficient training of diffusion models. 2023.

Jingfeng Yao, Wang Cheng, Wenyu Liu, and Xinggang Wang. Fasterdit: Towards faster diffusion transformers training without architecture modification. 2024.

Tianwei Yin, Michaël Gharbi, Richard Zhang, Eli Shechtman, Fredo Durand, William T. Freeman, and Taesung Park. One-step diffusion with distribution matching distillation. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) 2024*, 2023.

Huijie Zhang, Yifu Lu, Ismail Alkhouri, Saiprasad Ravishankar, Dogyoon Song, and Qing Qu. Improving training efficiency of diffusion models via multi-stage framework and tailored multi-decoder architecture. 2023.